

欧拉路径（图的一笔画问题）实验报告

一、问题描述：

本题要解决经典的“一笔画”问题，即在给定的无向图中寻找欧拉路径。市面上有很多此类小游戏，本学期学习的离散数学课程里刚好也讲到了欧拉图和一笔画问题。这道题给定一个特定形状的房子图（包含 5 个顶点），程序需要从顶点 1 开始，通过 DFS 找出所有能够不重复地走完所有边的路径，除了对角线和 1-4, 2-4 之间无边外，其余均相连。最终需按照节点编号从小到大的顺序，输出所有符合条件的路径。

解题时尽可能多写了宏定义，使得不同的题目只需要修改少数几个地方就可以得到答案，保证了可扩展性。

```
const int N = 5; //总顶点数
const int edge = 8; //总边数
const int start_point = 1; //初始顶点
```

二、基本要求：

输入：无特定输入数据，图结构直接在程序中初始化。

输出：按节点编号递增顺序输出所有可能的欧拉路径。每一条路径占一行，由顶点编号序列组成。

约束：必须从顶点 1 出发；所有的边恰好只画一次；输出必须包含所有满足条件的路径。

三、数据结构：

```
int map[N + 1][N + 1]; //邻接矩阵存储图，下标从1开始
int path[edge + 1 + 1]; //存储当前路径，从1开始，8个顶点9条边再加1
```

(1) `int map[N + 1][N + 1]`为邻接矩阵，用于存储图的边信息。

`map[N + 1][N + 1] = 1` 表示顶点 i 和 j 之间有一条未被访问的边，

`map[N + 1][N + 1] = 0` 表示无边或边已被访问。

(2) `int path[edge + 1 + 1]`为路径数组。用于记录当前搜索路径上的顶点，最后打印路径的时候用到。

四、功能说明：

```
void dfs(int point, int step_count) { // point 为当前的点，step_count 为计数器，用来确定
    path[step_count] = point; // 每次一进入循环，先把路径数组对应位置存为该点
    // 递归的终止条件，达到 edge + 1 就输出路径并且退出，当然可以根据题目的打印需求改 for 循环
    if (step_count == edge + 1) {
        for (int i = 1; i <= step_count; i++) {
            cout << path[i];
        }
        cout << endl;
        return;
    }
    for (int v = 1; v <= N; v++) { // 从小到大遍历顶点
        if (map[point][v] == 1) { // 如果 u 和 v 之间有边且未被访问
            map[point][v] = 0; // 标记边为已访问（删除边）
            map[v][point] = 0;
            dfs(v, step_count + 1); // 递归
            map[point][v] = 1; // 恢复边，以便寻找其他路径
            map[v][point] = 1;
        }
    }
}
```

本题的算法是带有回溯的深度优先搜索，整个程序行数不超过 50 行，重点依然在于递归的利用。递归函数 `void dfs(int point, int step_count)` `point` 为当前的点，`step_count` 为计数器，用来确定递归终止条件。进入函数第一句话 `path[step_count] = point` 将当前点存入路径。然后确定递归的两大要素：

1. 终止条件：当 `step_count == edge + 1` 时，说明找到了一条完整的欧拉路径。此时按照题目要求打印 `path` 数组中的序列，并返回。
2. 递归步骤：利用 `for` 循环从 1 到 `N` 遍历所有可能的下一个点 `v`，

按从小到大的顺序自然满足题目按节点递增顺序输出的要求，然后对 $\text{map}[\text{point}][v] == 1$ （说明有未访问的边）进行提示的操作，先标记 $\text{map}[\text{point}][v]$ 和 $\text{map}[v][\text{point}]$ 为 0，表示这条边已被使用。然后递归调用 $\text{dfs}(v, \text{step_count} + 1)$ 进行下一层搜索。最后递归返回后，将 $\text{map}[\text{point}][v]$ 和 $\text{map}[v][\text{point}]$ 重新置为 1，以便在其他路径搜索中再次使用该边。

```
int main() {
    for (int i = 1; i <= N; i++) { // 初始化邻接矩阵
        for (int j = 1; j <= N; j++) {
            if (i != j) map[i][j] = 1; // 先将所有元素除了本身置为1，全连接
            else map[i][j] = 0;
        }
    }
    // 本题只需要去除以下两组不存在的边，根据不同的题目修改即可
    map[1][4] = map[4][1] = 0;
    map[2][4] = map[4][2] = 0;
    dfs(start_point, 1); // 从给定顶点start_point开始搜索，当前是路径的第1个点

    return 0;
}
```

main 函数完成邻接矩阵初始化工作，本题 edge 数多于最大可能数的一半，所以先标记为 1 再把题目的少数不存在的边去除即可，随后调用 dfs 函数即可。

四、算法复杂度分析：

1. 时间复杂度：

该算法本质上是求解所有可能的边排列组合。对于一个度数为 D 的图，DFS 的最坏时间复杂度接近 $O(D^E)$ ，其中 E 是边的数量。在本题中，顶点最大度数为 4，边数为 8，虽然理论上是指数级，但由于图的规模非常小且存在大量剪枝（即边被访问后不可再次访问），实际计算量很小，程序可以瞬间运行完毕。

2. 空间复杂度：主要消耗在于邻接矩阵和递归栈。邻接矩阵 $O(N^2)$ ，本题中为 5×5 。路径数组与递归栈：深度最大为 $E + 1$ （即9层），空间复杂度为 $O(E)$ 。综上，空间复杂度极低，属于常量级。

五、总结和体会

本实验主要收获包括再次熟悉了邻接矩阵的用法，对于稠密图或小规模图，邻接矩阵非常直观且易于操作；学会了如何在DFS中处理边的消耗与恢复。与普通的图遍历（访问顶点后标记 $vis[u]=1$ ）不同，一笔画问题是访问边，因此需要动态修改邻接矩阵并在递归返回后进行回溯，这加深了我对递归的理解。

本题也可以考虑用欧拉路径存在的充要条件来进一步优化算法（对于无向图，度数为奇数的顶点个数必须为0或2）。本图中，顶点1的度为3，顶点2的度为3，其余为偶数，因此必然存在从1出发到2结束的欧拉路径。

六、附录

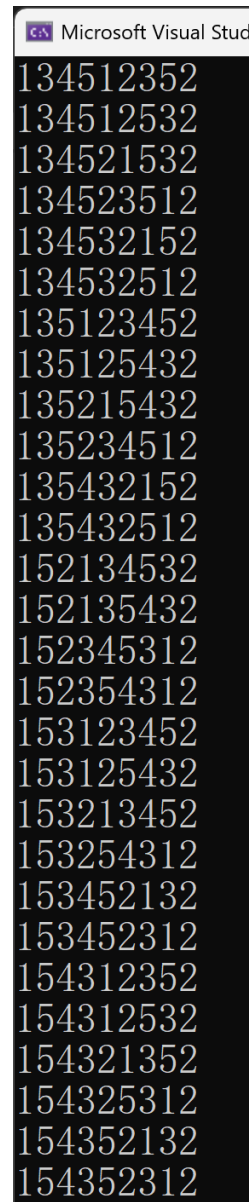
```
#include <iostream>
using namespace std;
const int N = 5; //总顶点数
const int edge = 8; //总边数
const int start_point = 1; //初始顶点
int map[N + 1][N + 1]; //邻接矩阵存储图，下标从1开始
int path[edge + 1 + 1]; //存储当前路径，从1开始，8个顶点9条边再加1
void dfs(int point, int step_count) { //point为当前的点，step_count为计数器，用来确定递归终止条件
    path[step_count] = point; //每次一进入循环，先把路径数组对应位置存为该点
    //递归的终止条件，达到edge + 1就输出路径并且退出，当然可以根据题目的打印需求改for循环的内容
    if (step_count == edge + 1) {
        for (int i = 1; i <= step_count; i++) {
            cout << path[i];
        }
        cout << endl;
        return;
    }
    for (int v = 1; v <= N; v++) { //从小到大遍历顶点
        if (map[point][v] == 1) { //如果u和v之间有边且未被访问
            map[point][v] = 0; //标记边为已访问（删除边）
            map[v][point] = 0;
            dfs(v, step_count + 1);
            map[point][v] = 1;
            map[v][point] = 1;
        }
    }
}
```

```

        dfs(v, step_count + 1); // 递归
        map[point][v] = 1; // 恢复边，以便寻找其他路径
        map[v][point] = 1;
    }
}
}
int main() {
    for (int i = 1; i <= N; i++) { // 初始化邻接矩阵
        for (int j = 1; j <= N; j++) {
            if (i != j) map[i][j] = 1; // 先将所有元素除了本身置为 1，全连接
            else map[i][j] = 0;
        }
    }
    // 本题只需要去除以下两组不存在的边，根据不同的题目修改即可
    map[1][4] = map[4][1] = 0;
    map[2][4] = map[4][2] = 0;
    dfs(start_point, 1); // 从给定顶点 start_point 开始搜索，当前是路径的第 1 个点

    return 0;
}

```



Microsoft Visual Studio

```

134512352
134512532
134521532
134523512
134532152
134532512
135123452
135125432
135215432
135234512
135432152
135432512
152134532
152135432
152345312
152354312
153123452
153125432
153213452
153254312
153452132
153452312
154312352
154312532
154321352
154325312
154352132
154352312

```